Week 2 Simple hash join

Radix hash join project

We implemented a simple hash join and measured how its build and probe phases behaved as the inputs grew. This gives us a working baseline for comparing radix hash join in Week 3. At two million rows per input, the median total time was 225.787 ms. Probing took the larger share of the measured execution time.

How the join works

Each record has a join key and a row ID. The key decides which records match. The row ID identifies the original row. For example, if the left keys are [7, 7, 9] and the right keys are [7, 9], the join produces three matching pairs. Both left rows with key 7 match the right row with key 7.

Build. We create a hash table from the left input. The hash function chooses a bucket for each key. When several records land in the same bucket, we link their array positions in a chain. This also lets the table keep repeated keys.

Probe. We read each right record, find its bucket, and check the keys in that chain. Every equal key adds a match. The benchmark counts the pairs and adds a checksum based on their row IDs. This makes the measured output consistent for later join comparisons.

How we tested it

The small tests compare the complete set of matching row pairs with a separate nested-loop join, which checks every possible pair. All 107 cases passed. They include empty inputs, no matches, repeated keys, keys in the same bucket, and boundary key values. The same tests passed with AddressSanitizer and UndefinedBehaviorSanitizer. A sanitizer benchmark at 10,000 and 100,000 rows per input also passed.

For the recorded benchmark, the left keys run from 0 to N-1 and the right keys are the even numbers from 0 to 2(N-1). Both inputs have unique keys and are shuffled with a fixed seed. This gives exactly N/2 matching pairs for the tested sizes. Every run checks the count and expected row-pair checksum.

What we measured

We used five sizes from 10,000 to two million rows in each input. Each size had one warmup run followed by seven recorded runs. Each run built a fresh hash table. Build time includes allocating the table, setting up its buckets and inserting the left records. Probe time includes lookups, match counting and the checksum. Input generation, result checking, file writing and freeing the table happen outside the timer.

The test ran in Ubuntu under WSL2 on an AMD Ryzen 7 7840HS. We used GCC 11.4.0 with -O2 and kept the thread on logical processor 0. A record uses 8 bytes. The table has at least twice as many buckets as left records, so later comparisons should keep that setting consistent.

What changed as the inputs grew

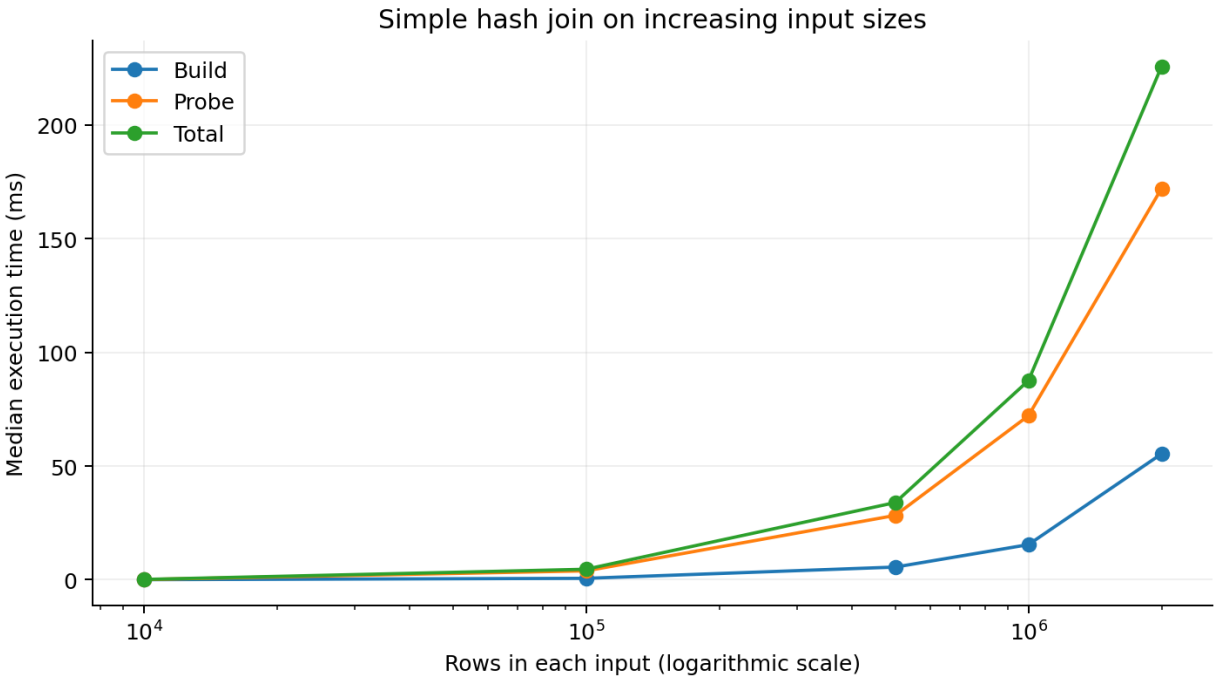


Figure 1. Median times across seven runs at each size. Each input has the row count shown on the horizontal axis. That axis uses logarithmic spacing.

The probe line stays above the build line at every size. From 500,000 to one million rows per input, median total time rises from 33.948 to 87.570 ms. At two million rows it reaches 225.787 ms. The gap between the build and probe lines becomes larger as the inputs grow.

Rows in each input	Matches	Build ms	Probe ms	Total ms
10,000	5,000	0.038	0.097	0.138
100,000	50,000	0.618	3.904	4.621
500,000	250,000	5.575	28.295	33.948
1,000,000	500,000	15.440	72.130	87.570
2,000,000	1,000,000	55.384	172.134	225.787

The table uses medians. Each phase median is calculated separately, so build and probe medians can add up to a slightly different value from the total median.

Part of the increase is expected because larger inputs need more insertions and lookups. The time per input row also rises. Dividing median total time by the number of right rows gives about 14 ns at 10,000 rows and 113 ns at two million. This includes both phases. It suggests that processing each row becomes more expensive as the test grows.

The estimated space for the two inputs and hash-table arrays rises from 0.48 MiB to 77.78 MiB. A larger memory footprint may contribute to the slowdown. The bucket count also changes with input size. To separate these effects, we will need the partitioned join comparison and, where available, hardware counters.

Where the join spends its time

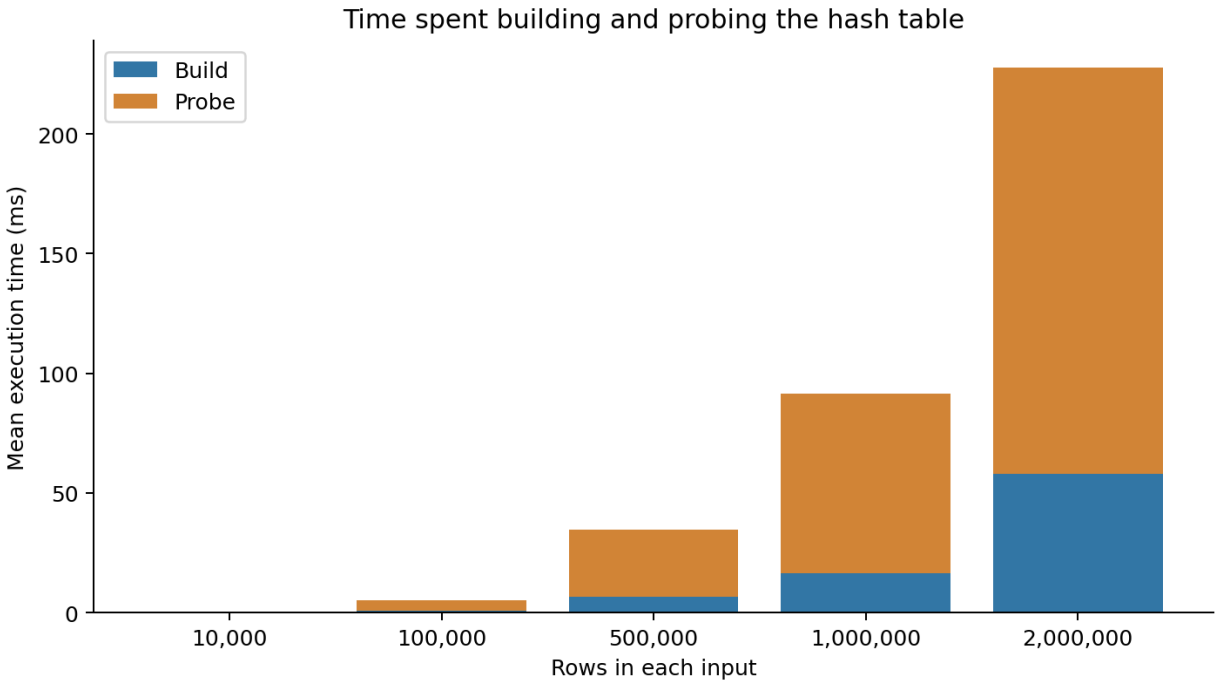


Figure 2. Mean build and probe times. Each full bar is the mean total time for that input size. The orange section shows the probe phase.

At two million rows per input, the mean build time is 57.978 ms and the mean probe time is 169.658 ms. Probing accounts for about 74.5% of the mean total. The larger orange sections show that searching for matches takes more time than building the table in these runs.

During probing, consecutive input keys can lead to different buckets spread across the table. Some lookups also follow a chain. Those accesses are a possible reason for the probe cost. The current measurements combine hashing, key comparisons, memory access, match counting and checksum work. We have measured the full phase, so we should test which parts improve after partitioning.

How this guides Week 3

We will split both inputs into partitions using the same hash bits, then join corresponding groups. Each local join will work with a smaller hash table. This gives the table a better chance of staying in cache while its keys are used.

We will keep the generated inputs, match rate, hash function and match-counting output consistent. We will measure partitioning, local builds and local probes separately. The full radix join time will include all three phases. Then we can see whether faster local joins save enough time to cover partitioning, and how that changes with input size.

This completes the Week 2 baseline. The source and running instructions are in hash_join.c and README.md. The individual measurements, run settings, summary statistics and two plots are saved in the results folder. The saved run uses unique keys with a 50 percent match rate. Duplicate handling was checked separately in the correctness tests.